Project Architecture

2026-10-08

1. The project

We're building a low-cost queue system for a busy campus canteen in Kerala. The idea is to keep two very different kinds of service apart:

- The **standard meal line** (about 85% of customers) is fast — rice, curries, side dishes, served in 5–10 seconds each. We call this **Counter A**.
- **Short orders** (about 15%) — porotta, chicken curry, snacks — need someone to assemble and tray them. That happens in the **kitchen**, and customers collect at a separate **Counter B**.

Three ESP32 stations sit at those three places. They show the current token, chime when there's something new, and record a button press when staff hand food over. Everything else — money, queues, order state — lives on one central backend.

Hardware

- 3× ESP32-C3 development boards
- 3× 16x2 LCD displays with I2C backpack (address `0x27` or `0x3F`)
- 3× active buzzer modules
- Pushbuttons, read with `INPUT_PULLUP` and debounced in software
- Breadboards, jumper wires, 3× 5V USB adapters and cables

Two things the BOM doesn't cover yet: the USB adapters and cables still need to be procured, and the C3 boards ship with **unsoldered** headers — solder those before anything goes on a breadboard.

The whole system runs on one local Wi-Fi router. MQTT runs on a local Mosquitto broker (Docker, on the LAN). No cloud anywhere — if the internet drops, the canteen keeps serving.

The three stations

- **Node 1 — Kitchen Assembly Bay.** Watches incoming short orders, shows the token and what's in it, and staff press a button once the tray is packed.
- **Node 2 — Counter A, the rapid meals line.** Shows who's being served right now and how many are waiting. Staff press the button as each customer is handed over. This line never touches the kitchen.
- **Node 3 — Counter B, specials pickup.** Lights up when a tray from Node 1 is ready, chimes, and waits for staff to confirm the handover.

Design rules everyone follows

- The **token number is just `orders.id`** — no separate counter table.
- An order is **`pending` until the food is physically handed over.** Nothing gets marked complete when it's merely *published* or *ready*.

- **The backend owns every queue.** The ESP32s are thin: a display, a button, and a chime. They only ever receive payloads the backend has already validated.
- **All money is in paise** (`int64`), and the balance is debited atomically inside `CreateOrder` before anything is sent to MQTT.

---

2. How everything fits together

Placing an order

An order comes in through `POST /orders` (from the kiosk, or `curl` during testing). `CreateOrder()` writes the order, its line items, and the balance debit in a single transaction. If anything fails, nothing is written.

The backend then looks at the item codes and picks a queue:

- Order contains **only `MEAL`** → it joins the Counter A queue.
- Order contains **anything else** → it joins the Kitchen queue as one token, with a short code taken from the first non-MEAL item (at most 7 characters — `POR`, `CHK`, `SNACK`). Mixed carts go through the kitchen too, so a combined order stays one tray and one handover.

The fast path — standard meals

1. The backend publishes the head of the Counter A queue: `canteen/counter-a/now` → `105 Q44`. This message is **retained**.
2. Node 2 shows `NOW:105` and `Q:44`, and beeps.
3. Staff serve that customer, then press the button. Node 2 publishes `canteen/counter-a/served 105`.
4. The backend marks order 105 as completed and publishes the next head.
5. Repeat until the queue is empty.

If a `served` message arrives twice (QoS 1 retries do this), the second `CompleteOrder` hits `ErrConflict`, which the backend treats as "already done" — it just republishes the current head. This is the dedup mechanism; no extra bookkeeping needed.

The specials path — kitchen to Counter B

1. The backend publishes `canteen/kitchen/now` → `106 POR Q3`. Node 1 shows `TK:106 [POR]`, `Q:3`, and beeps.
2. Staff pack the tray from the warmers and press the button. Node 1 publishes `canteen/kitchen/done 106`.
3. The backend moves that token from the kitchen queue to the ready list and publishes `canteen/counter-b/ready` → `106 POR W2`. Note: **no** `CompleteOrder` here — the food is ready, but nobody has collected it yet.
4. Node 3 shows `RDY:106 [POR]`, `W:2`, and chimes.
5. Staff hand the tray over, check the token, and press the button. Node 3 publishes `canteen/counter-b/collected 106`.
6. Only now does the backend mark the order completed, and it publishes the next ready token.

What happens during the lunch rush

Forty-five students order `MEAL` between 11:55 and 12:05. All 45 sit in the queue as `pending` orders. The LCD never needs to show all of them — it shows the head and the depth (`NOW:105 / Q:44`), so the line itself can see how long the wait is.

Serving at 5–10 seconds each, the queue drains in roughly 4–7 minutes. If a node reboots mid-rush, the retained `now` message restores the head as soon as it reconnects.

---

3. MQTT

One local Mosquitto broker, QoS 1 on every topic, plain-ASCII payloads. No JSON — the payloads have to fit on a 16-character LCD anyway, and skipping JSON means no ArduinoJson on the ESP32.

Topic	Direction	Retained	Payload	Example
`canteen/counter-a/now`	backend → Node 2	yes	`<head>` `Q<depth>`, or `IDLE Q0`	`105 Q44`
`canteen/counter-a/served`	Node 2 → backend	no	`<id>`	`105`
`canteen/kitchen/now`	backend → Node 1	yes	`<head> <code>` `Q<depth>`, or `IDLE Q0`	`106 POR Q3`
`canteen/kitchen/done`	Node 1 → backend	no	`<id>`	`106`
`canteen/counter-b/ready`	backend → Node 3	yes	`<head> <code>` `W<n>`, or `NONE W0`	`106 POR W2`
`canteen/counter-b/collected`	Node 3 → backend	no	`<id>`	`106`
`canteen/<node>/online`	node → all (LWT)	yes	`1` or `0`	`1`

A few notes:

- The three **display topics are retained** so a rebooting node recovers its state without anyone re-sending. The three **ack topics are transient** — if a node was offline it simply missed that one event, and the status check handles it.
- `W<n>` on Counter B is the number of trays waiting, `Q<n>` on the other two is the number of customers waiting.

Backend side

`mqtt.go` in `food-kiosk/`, using `github.com/eclipse/paho.mqtt.golang`:

- One `Publish(topic, payload)` helper.
- One subscriber that dispatches the three ack topics to the right handler (`served` → complete + advance Counter A; `done` → move to ready list; `collected` → complete + advance Counter B).

This is testable before any hardware exists:

```
mosquitto_sub -t 'canteen/#'
```

---

4. Backend APIs

Already built and tested (in `MC-Project-Backend/food-kiosk/database/`): `CreateOrder`, `CompleteOrder`, `CancelOrder`, `TopUpUser`, `ListOrdersByStatus`, `GetOrderByID`. These don't need changes.

Still to implement — `main.go` currently only opens the database, so the HTTP server and the MQTT wiring are the two missing pieces:

```
POST /users            {roll_no, name, phone}        -> {id}
POST /users/:id/topup  {amount_paise}                -> {balance_paise}
POST /orders           {user_id, lines: [{code, quantity}]} -> {id, total_paise}
GET  /queue            ?lane=a|kitchen|ready         -> {head, depth, items:
[ids]}
POST /orders/:id/cancel                              -> {balance_paise}
```

- `POST /orders` also performs the queue routing and publishes the new head (or the updated depth) described in section 2.
- `GET /queue` reads `ListOrdersByStatus("pending")` filtered by lane. It's for debugging and admin — the nodes never call it; everything they show comes from MQTT.
- Acknowledgements come back over MQTT, not over HTTP.

---

5. Hardware

There are three people on firmware, but there should be **one sketch**, not three. The stations share almost everything — Wi-Fi, MQTT, LCD, buzzer, button — so the only difference is a role flag at the top:

```
#define ROLE KITCHEN   // or COUNTER_A or COUNTER_B
```

Libraries

Just two: `LiquidCrystal_I2C` and `PubSubClient`. Button debouncing (~30 ms stable-state filter with edge detection) and the beep (150 ms on, then off) are about ten lines of `millis()` each — no extra libraries.

Wiring (ESP32-C3)

Signal	Pin
LCD SDA	GPIO5
LCD SCL	GPIO6
Buzzer	GPIO7
Button	GPIO10 (`INPUT_PULLUP`, other leg to GND)

- **Avoid GPIO8 and GPIO9** — they're strapping pins.
- **Avoid GPIO18 and GPIO19** — they're the USB-CDC pins; using them kills your serial monitor.
- Run a quick I2C scanner on first boot to confirm the LCD address (`0x27` vs `0x3F`) before hardcoding it.

How the loop works

`loop()` does three things and never calls `delay()`:

1. Keep Wi-Fi alive — reconnect on a 5-second `millis()` backoff if it drops.
2. Keep MQTT alive — reconnect the same way, and call `client.loop()` every pass.
3. Read the button (debounced) and service the beep (turns itself off when its time is up).

The LCD is only written to when the displayed state actually changes, so `client.loop()` keeps running thousands of times per second and stays responsive.

What each role does

Role	Subscribes to	Button press	LCD line 1 / line 2
Kitchen	`kitchen/now`	publishes `kitchen/done <shown>`, clears its screen	`TK:106 [POR]` / `Q:3`
Counter A	`counter-a/now`	publishes `counter-a/served <shown>`	`NOW:105` / `Q:44`
Counter B	`counter-b/ready`	publishes `counter-b/collected <shown>`, clears its screen	`RDY:106 [POR]` / `W:2`

Behaviour rules:

- Beep on every new MQTT message, and once more on a button press.
- Empty queue shows `IDLE` or `NONE` — no beep, and the button does nothing.

Bring up **Counter B first**: it's the simplest flow (subscribe → display → beep → button clears), but it exercises every shared subsystem. Then Kitchen, then Counter A.

---

6. Build order

1. **Broker + backend MQTT.** Get Mosquitto running, wire up `Publish` and the subscriber, and watch everything with `mosquitto_sub -t 'canteen/#'`. No hardware required.
2. **One node, then the rest.** Counter B → Kitchen → Counter A.
3. **End-to-end test.** `curl POST /orders` → kitchen LCD lights up → button → Counter B chimes → button → balance settled.
4. **Resilience test.** Kill the router mid-queue. All three nodes must reconnect on their own, the retained display messages must restore correctly, and duplicate acks must be deduplicated.

Deliberately out of scope (for now)

- **Kiosk web UI** — `curl` is enough for the demo.
- **Daily-resetting tokens** — `orders.id` is unique and fits; revisit only if staff ask for a counter that restarts each day.
- **Cloud broker** — local only; the canteen shouldn't depend on the internet.
- **OTA firmware updates** — flash over USB.
- **Any HTTP beyond the five routes above.**